

Enjoy! Java Learningシリーズ

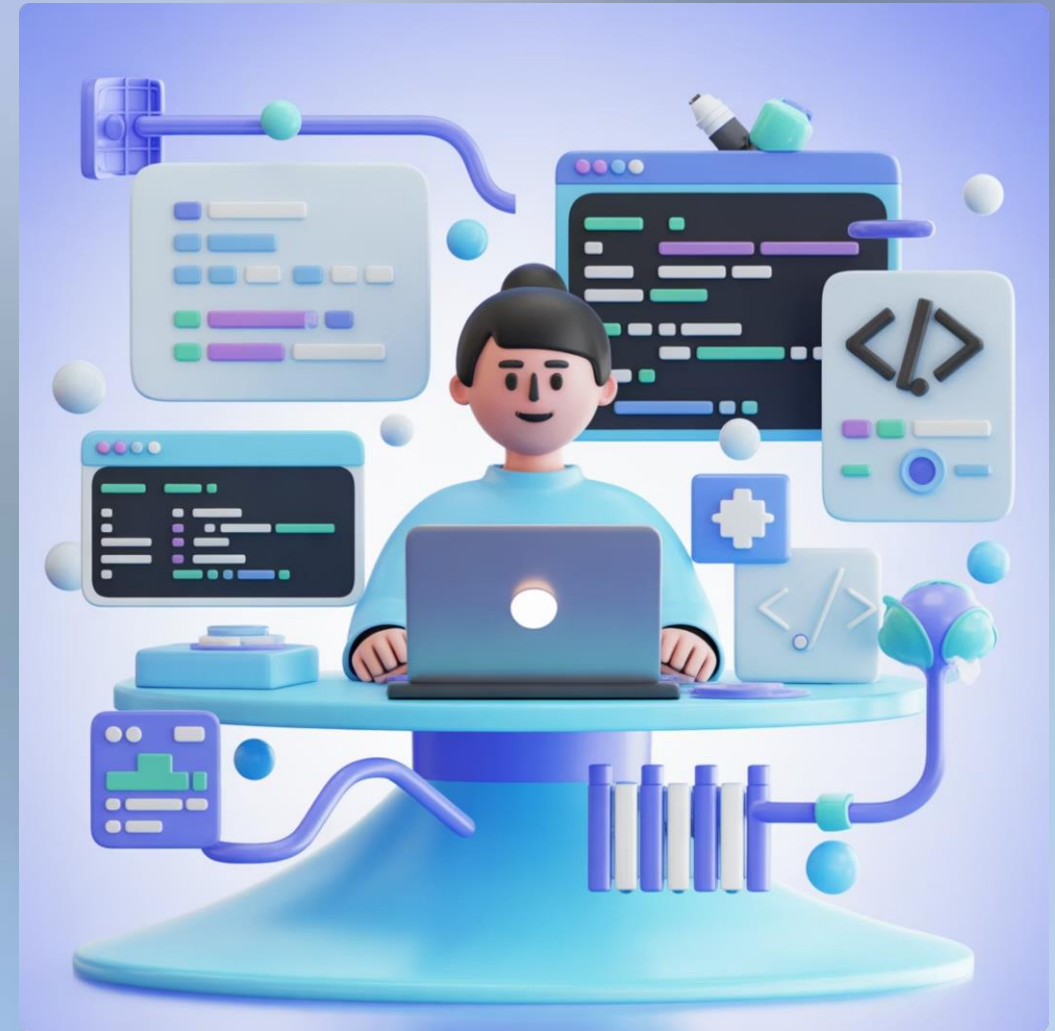
『ゼロから始めるJavaプログラミング』

# Javaプログラミング基礎編

本講座の目標：

Javaプログラミングの基本的な「読み・書き」をできるようになること。単なる暗記ではなく、「なぜそうするのか」を理解し、自走できるエンジニアの第一歩を踏み出すことを目指します。

プログラミングは難しそうに見えますが、一步一步進めば必ず理解できます。このシリーズを通じて、コンピュータと対話する楽しさを体験していきましょう。



# シリーズの全体構成

『ゼロから始めるJavaプログラミング』は、初心者から中級者へ徐々にスキルアップできるように構成されています。段階的に学ぶことで、無理なく実力をつけていくことができます。

## 1 Javaプログラミング基礎編

プログラミング初心者のため、他の言語とも共通するJavaプログラミングの構造や考え方について学びます。変数、制御文、メソッドなど、すべてのプログラミング言語に通じる基本的な概念を習得します。

1

- 環境構築からスタート
- 基本文法の理解
- プログラムの流れを制御
- コードの整理と再利用

## 2 Javaオブジェクト指向プログラミング編

Javaの最も強力な特徴であるオブジェクト指向プログラミング(OOP)と、Javaの特徴的な機能について学びます。クラス、継承、ポリモーフィズムなど、より高度な概念に挑戦します。

2

- クラスとオブジェクトの理解
- 継承とインターフェース
- カプセル化とポリモーフィズム
- 実践的な設計パターン

## 3 RPGに学ぶJavaプログラミング※

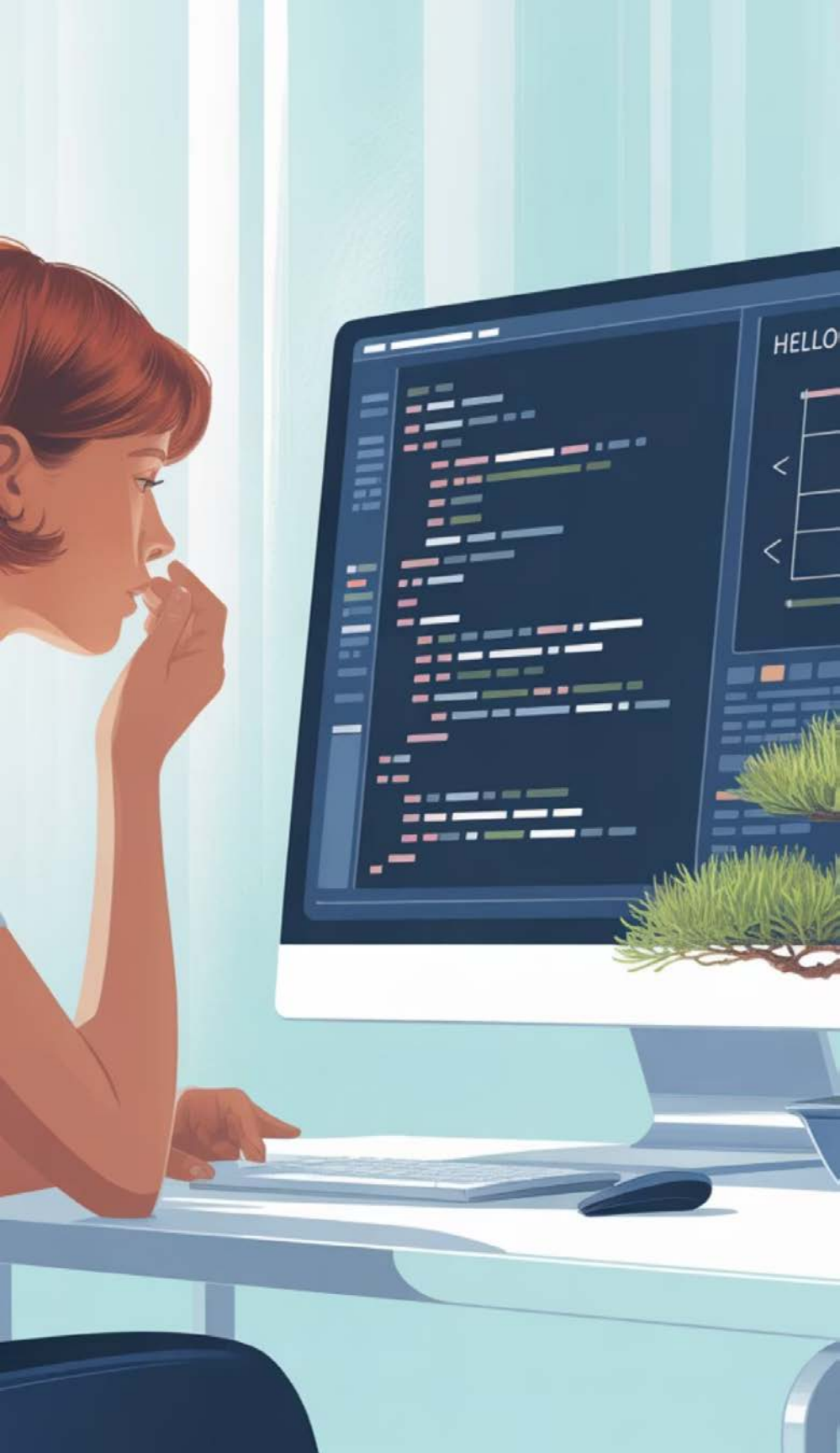
ゲーム開発を通じて、楽しみながらプログラミングスキルを向上させることができます。

3

- RPGゲームの全体像を掴む
- データ駆動とMVCモデル
- 様々なクラス構成（アイテム、キャラクター、会話）
- 全体マップとイベント・ストーリー

📖 ※1『RPGに学ぶJavaプログラミング』は、別教材実践編です。





# この教材の特徴

## ”分かる”を重視した構成

このビデオ教材はJavaの基本となる文法について、単に細かく覚えるのではなく、初心者がつまずきそうな重要な考え方を「なぜ?」や「どうして?」という疑問に答えながら理解できる構成になっています。

副読本には、文法について網羅的に詳しく記載があります。ぜひ参照してください。また、コードサンプルについても記載がありますので、それらを参考に自分でタイピングして実際に動かしてみることをお勧めします。



### 学習のポイント

- 暗記ではなく理解を重視
- 実際に手を動かして学ぶ
- 段階的にスキルアップ
- 質問を大切にする姿勢

### 効果的な学習方法

- ビデオを見ながらコードを書く
- エラーを恐れずに試す
- わからないことは調べる
- 繰り返し練習する

# Javaオブジェクト指向プログラミング編

## 学習カリキュラム

0

### 第0章 Javaの世界へようこそ！

環境構築とプログラムの実行

1

### 第1章 プログラミングの第一歩

Hello Java World!

2

### 第2章 データを入れる魔法の箱

変数とデータ型

3

### 第3章 計算と比較はお任せ！

演算子と型キャストメソッド

4

### 第4章 場合分けを考えよう！

条件分岐 if文

5

### 第5章 もう一つの選択肢！

スマートな分岐 switch文

6

### 第6章 面倒な作業は自動化！

ループ処理 for, while文

7

### 第7章 コードを整理整頓

再利用できる部品「メソッド」

8

### 第8章 チュートリアル

ダイヤモンド、Nクイーン、迷路問題

これらの章を通じて、プログラミングの基礎を学びます。  
焦らず、一つ一つの概念をしっかりと理解しながら進んでいきましょう。



# Javaで出来ること



Java言語は、C、Ruby、Python言語と同じような汎用言語の一つです。  
さまざまなプログラムを作成することができ、その汎用性の高さから世界中の企業で採用されています。



## CUIアプリケーション

文字情報を使用したコマンドラインベースのプログラム。サーバー管理やデータ処理に活用されます。



## Webアプリケーション

サーバ上で動作する、ブラウザからアクセスできるシステム。ECサイトや業務システムなど幅広く利用されています。



## 通信モジュール

ミドルウェアとしてシステム間の通信を担当する重要な部品。企業の基幹システムを支えています。



## GUIアプリケーション

ウィンドウシステムを利用した視覚的なデスクトップアプリケーション。ユーザーフレンドリーな操作が可能です。



## Androidアプリ

スマートフォンやタブレットで動作するアプリケーションやゲーム。世界中のユーザーに届けられます。



## バッチ処理

大量のデータを一括で処理するモジュール。夜間処理など、効率的なデータ管理に不可欠です。

# 第0章 Javaの世界へようこそ!

## 最初の一步:環境構築とプログラムの実行

この章の目標は、Javaプログラミングを始めるための準備を整え、ソースコードを書いてから実行するまでの一連の流れを理解することです。

プログラミングの旅は、まず道具を揃えることから始まります。



# Java 開発キット

Javaプログラムを動かすには、JDK(Java Development Kit)というJava開発キットが必要です。これは、Javaプログラムを作成するためのツールボックスのようなもので、コンパイラと実行環境が入っています。まずはこれを手に入れましょう！

## JDKをインストール

JDKは無料でダウンロードできます。Oracle社の公式サイトからインストールしましょう。インストール後は、環境変数の設定も忘れずに行ってください。



### コンパイラ

人間が書いたJavaのソースコードを、コンピュータが理解できる言葉(.classファイル)に翻訳してくれます。通訳のような役割を果たします。



### 実行環境

翻訳された.classファイルを実際に動かしてくれます。プログラムを実行するための舞台を提供します。



# プログラムの実行方法

## ① コマンドライン(基本)

一つはコマンドラインから、JDKにあるコンパイラツール（`javac`コマンド）と実行コマンド（`java`コマンド）で、一つずつ順を追って実行する方法です。

01

### 書く

テキストエディタでプログラムのソースコードを書き、`App.java`という名前で保存します。

02

### 翻訳する

コマンドラインで`javac App.java`と入力し実行（コンパイル）します。`App.class`という実行ファイルが作られます。

03

### 実行する

コマンドラインで`java App`と入力します。  
プログラムが実行されます。

## ② 統合開発環境(IDE)(推奨)

実際の開発では、IDEという専用のソフトウェアを使うのが一般的で、コードを書くための高機能なエディタ、コンパイラ、実行・デバッグツールなどが一つになったプログラミングの万能ツールボックスです。

### 主なIDE

- Eclipse:オープンソースで人気
- IntelliJ IDEA:強力な機能を持つ
- Visual Studio Code:軽量で拡張性が高い

### メリット

面倒なコンパイルや実行は、再生ボタン  を押すだけで自動でやってくれます。コードの間違いもリアルタイムで教えてくれます。







# 第1章 プログラミングの第一歩

Hello Java World!

この章の目標は、ソースコード上でJavaプログラムがどのような構造で動いているのか、その全体像を掴むこと。

そして、お決まりの「Hello World!」を自分の手で動かし、プログラミングの楽しさを体感することです。すべてのプログラマが通る最初の一步を、一緒に踏み出しましょう。

# プログラムの基本構造を理解しよう

Javaのプログラムは、クラス(Class)という部品の集まりでできています。  
まずは、プログラムを実行するための「玄関」となる特別な場所、mainメソッドを覚えるところから始めましょう。

```
public class Main {  
    // プログラムはここから始まる!  
    public static void main(String[] args) {  
        // ここに実行したい命令を書く  
        .....  
    }  
}
```

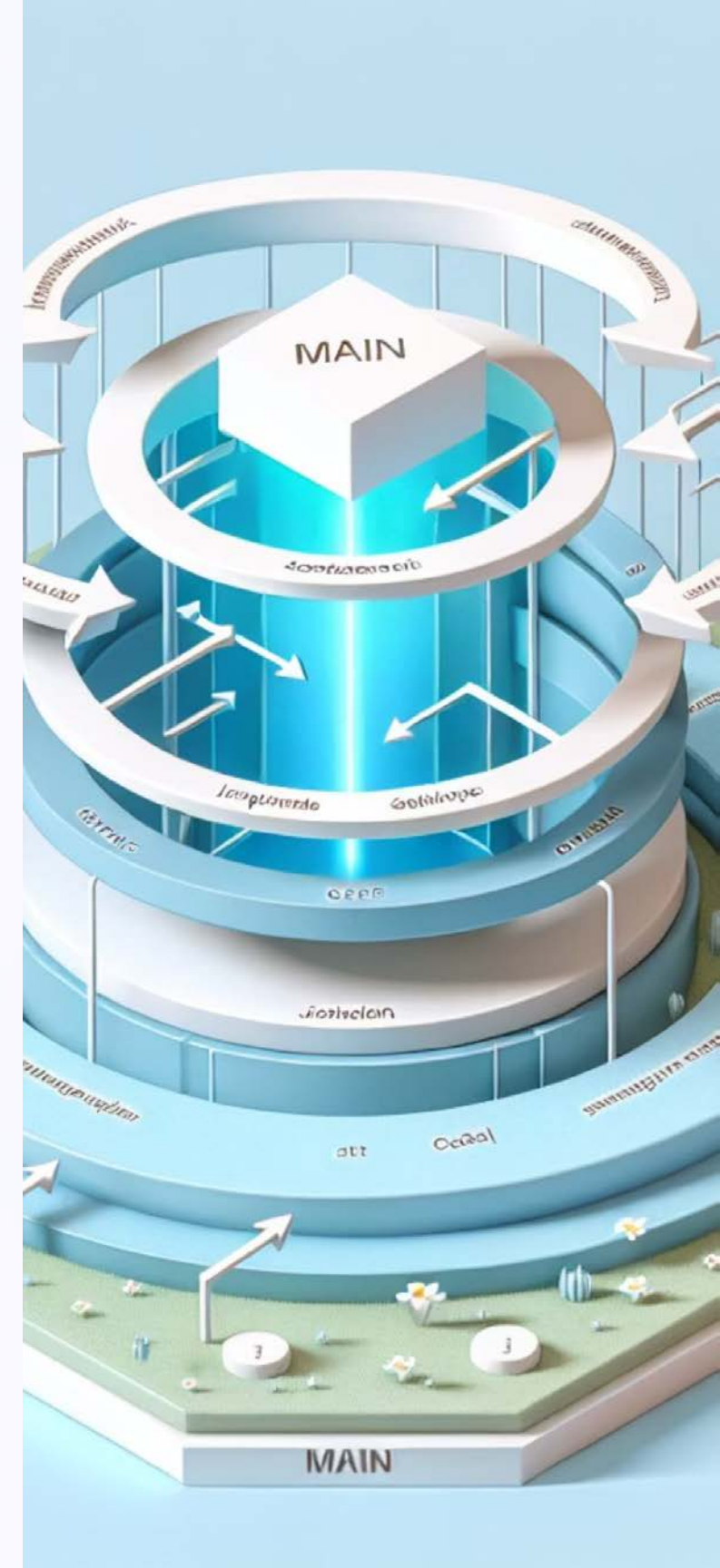
**class Main { ... }**

**Main**という名前のクラス(部品)を定義しています。家に例えると、建物全体の設計図のようなものです。Javaのプログラムは必ずこのクラスが1つ以上必要です。

**main(...) { ... }**

プログラムが起動したときに、最初に実行される特別なメソッド(処理の塊)です。プログラムの「玄関」であり、すべてはここから始まります。

- プログラミングは料理に似ています。クラスはキッチン、mainメソッドは調理開始のゴングのようなものです。まずはこの構造に慣れましょう。





# 最初のプログラム

## "Hello World!"

```
public class Main {  
    // プログラムはここから始まる!  
    public static void main(String[] args) {  
        System.out.println("Hello World!");  
    }  
}
```

実行結果:

```
Hello World!
```

- `System.out.println(...)` は、括弧の中身を画面に表示するための命令です。これは「プリントライン」と読みます。
- 命令の最後には必ず**セミコロン(;)**を付けます。これを文(Statement)と呼びます。句点のようなものだと考えてください。
- **ダブルクォーテーション("")**で囲まれた部分が、実際に表示される文字列です。この中身を変えれば、好きなメッセージを表示できます。



# 文(Statement)とコメント

## 文(Statement)とは?

プログラムは、コンピュータに対する命令の集まりです。Javaでは、その一つ一つの命令を**文(Statement)**と呼び、文の終わりには必ず**セミコロン(;)** を付けます。

```
// これは一つの文
System.out.println("Hello!");
```

```
// これも一つの文
int price = 100;
```

- ❏ セミコロンを忘れると、文の終わりが分からずにエラーになります。よくある間違いなので気を付けましょう!

## コメントでメモを残そう

コメントとは、プログラムの動作に影響を与えない、人間のためのメモです。コードが何をしているのかを分かりやすくするために使います。

```
// 一行コメント
System.out.println("実行");//行末コメント
/*
 * 複数行コメント
 * ここにメモを書ける
 */
```

## コメントの種類

- `// ...` その行の最後までコメント
- `/* ... */` 複数行をまとめてコメント



## 第1章のまとめ

# プログラミングの第一歩

- Javaプログラムはクラス (Class) の中に記述する。
- プログラムの開始点は mainメソッド。
- `System.out.println("...")` は画面に文字を表示する命令。
- 一つ一つの命令を 文 (Statement) と呼び、最後にはセミコロン (;) が必要。
- コメント を使って、コードにメモを残すことができる。

## 第2章 データを入れる魔法の箱!

### 変数とデータ型

この章の目標は、プログラムで数値や文字といった「データ」を扱うための基本である、「変数」と「データ型」の概念を理解することです。

コンピュータのメモリ上にデータを保存し、それらを名前呼び出して利用する方法を学びます。変数は、プログラミングにおける最も基本的で重要な要素です。



# 『変数』ってなんだろう？

変数とは、データを入れておくための名前付きの「箱」のようなものです。  
プログラムは、この「箱」に必要な情報を一時的に保管し、後から名前を取り出して使ったり、中身を書き換えたりします。

1

## 箱を用意する

名前を付けて変数を宣言する

2

## 箱にデータを入れる

値を代入する

```
// 1. 変数の宣言（箱の用意）
```

```
int myNum;
```

```
// 2. 値の代入（箱にデータを入れる）
```

```
myNum = 10;
```

```
// 中身を表示してみる
```

```
System.out.println(myNum); // 結果: 10
```

```
// 宣言と代入を同時に行う
```

```
int myNum = 15;
```

```
System.out.println(myNum); // 結果: 15
```

# なぜ『データ型』が必要なの？

Javaでは、変数(箱)を用意するときに、どんな種類のデータを入れるための箱なのかをあらかじめ決めておく必要があります。これをデータ型といいます。

- 整数を入れる箱 (int)
- 文字を入れる箱 (char)
- 文章を入れる箱 (String)

なぜ？

データの種類によって、必要なメモリのサイズや扱い方が異なるためです。型を決めておくことで、コンピュータは効率よく動作でき、プログラマの間違いも減らせます。これはJavaの型安全性という重要な特徴です。

1+ 整数型  
int型

1.0 小数型  
double型

T 文字型  
char型



# 基本データ型を覚えよう

## 数値を扱う型

| データ型   | 説明              | 例           |
|--------|-----------------|-------------|
| int    | 整数の最も一般的な型      | 10, -5, 0   |
| double | 小数の最も一般的な型      | 3.14, -0.01 |
| byte   | 小さい整数(-128～127) | 100, -50    |
| long   | 大きい整数           | 9999999999L |
| float  | 軽量な小数           | 3.14F       |

## その他の基本型

### boolean

真偽値：「はい」か「いいえ」の2択

`true` または `false`

### char

1文字だけを扱う型： `'A'`, `'あ'`, `'1'`

シングルクォーテーション `'?'` で囲む

### String

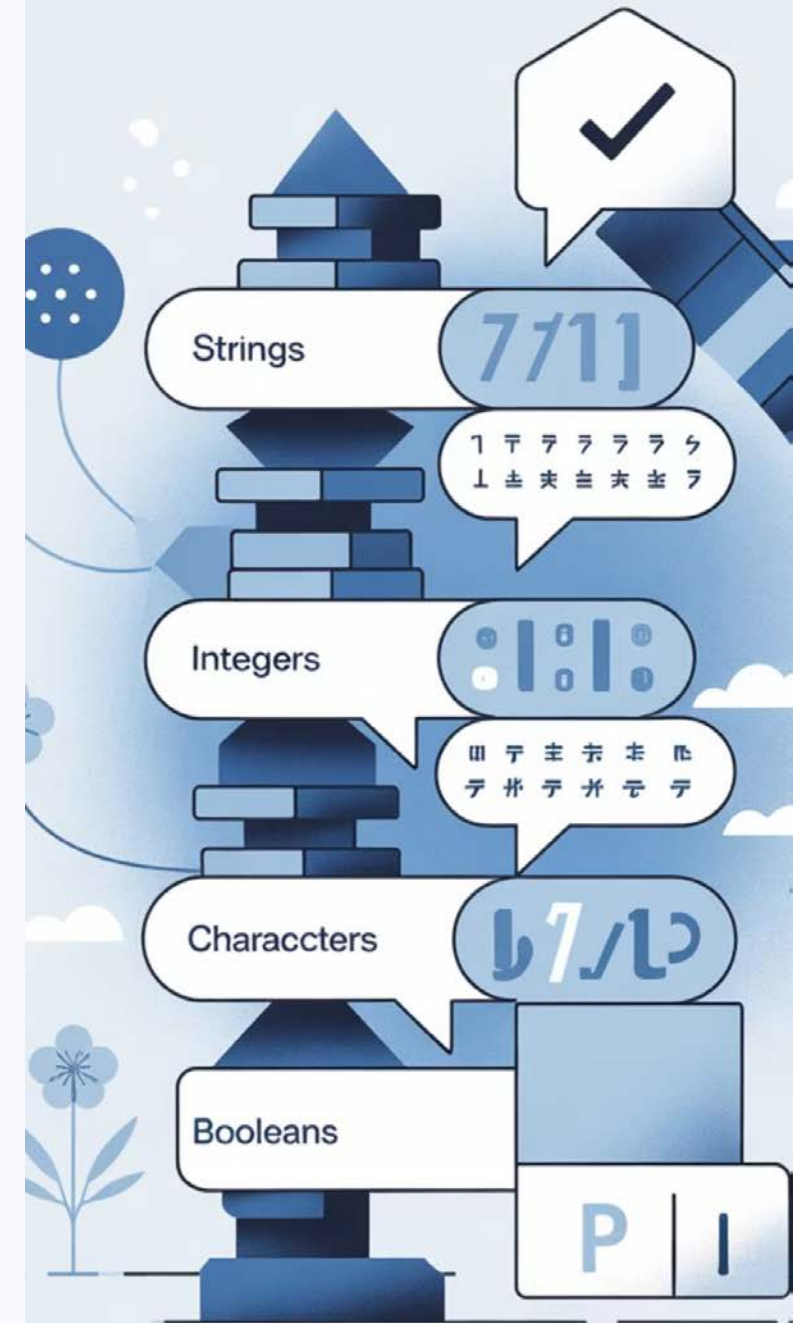
文字列(文章)型： `"Hello World!"`

ダブルクォーテーション `""` で囲む

❏ まずは**整数**なら**int**、**小数**なら**double**と覚えておけばOK!必要に応じて他の型も使っていきます。

# 変数を使ってみよう!

```
public class Variables {  
    public static void main(String[] args) {  
        // ----- 文字列型 -----  
        String name = "John";  
        System.out.println("My name is " + name);  
        // ----- 整数型 -----  
        int age = 20;  
        System.out.println("I am " + age + " years old.");  
        // ----- 小数型 -----  
        double height = 175.5;  
        System.out.println("My height is " + height + "cm.");  
        // ----- 文字型 -----  
        char bloodType = 'A';  
        System.out.println("Blood type is " + bloodType);  
        // ----- 真偽値型 -----  
        boolean isStudent = true;  
        System.out.println("Am I a student? " + isStudent);  
    }  
}
```



## 変数を使ってみよう!



### 文字の連結

+ 記号を使うと、文字列と変数を連結して表示できます。

```
int age = 20;
System.out.println("I am " + age + " years old.");
```



### 定数

プログラムの中で、一度値を入れたら絶対に変わらないようにしたい値には、**final** を変数型の前に付けて 定数 にします。

```
// final を付けると、もう値を変更できなくなる
final double PI = 3.14;
PI = 3.14159; // エラーになる！ (再代入できない)
final int DAYS_IN_WEEK = 7;
```



### 値の書き方（リテラル）

ソースコードに直接書く100や"Hello"のような値そのものをリテラルと呼びます。Javaには、リテラルの書き方にもルールがあります。整数リテラルは、**int**型として扱われます。また、小数リテラルは**double**型として扱われます。リテラルを他の型の値にしたい場合は、末尾に'L','F'をつけます

```
// 整数リテラルは、そのままint型変数に代入できますint i
= 123;
// 小数リテラルは、そのままdouble型変数に代入できます
double d = 3.14;
```

```
long l = 123456789012L; //'L'を付けて long型だと示す
float f = 3.14F; //'F'を付けてfloat型だと示す
```



# 配列とは？

## 変数とデータ型

同じ型のデータを、複数まとめて格納できるオブジェクトです。

番号付きの連続したロッカーをイメージしよう！

## 配列の重要なルール

1

同じデータ型しか入れられない

intの配列にはintしか入れられない。

2

サイズは後から変更できない（固定長）

最初に「10個のロッカー」と決めたら、後から11個に増やすことはできない。

3

各部屋には「インデックス」という0から始まる番号が付いている

10個のロッカーがあれば、番号は0番から9番まで。



# 配列を使ってみよう（その1）

変数とデータ型

## 配列の宣言・生成・初期化

書き方1：サイズを指定して、後から値を入れる

```
// int型の要素を3つ格納できる配列を生成
int[] scores = new int[3];
// インデックスを使って各要素に値を代入
scores[0] = 80;
scores[1] = 95;
scores[2] = 72;
```

書き方2：生成と同時に値を初期化する

```
// 中身のデータが最初から決まっている場合に便利
String[] names = {"Sato", "Suzuki",
                  "Takahashi"};
// 配列の長さ（要素数）の取得
// .length というフィールドで取得できる。（メソッドではないので()は不要！）
int num = scores.length; → 3
```

配列の注意点：そのまま表示しようとすると...？

```
int[] scores = {80, 95, 72};
System.out.println(scores);
```

実行結果（例） [I@15db9742これは、配列の中身ではなく、配列オブジェクトがメモリ上のどこにいるか、という種類やアドレスに近い情報が表示されてしまいます。

# 配列を使ってみよう（その2）

## 変数とデータ型

配列のループ処理、多次元配列（ループ処理、第6章でforループ処理は習います。）

### for文によるループ処理

```
int[] numbers = {10, 20, 30, 40, 50};
for (int i = 0; i < numbers.length; i++) {
    System.out.println("numbers["+i+"]="+numbers[i]);
}
```

### 2次元配列の宣言と生成

```
int[][] matrix = new int[3][4]; // 3行4列のint型2次元配列
// matrix[行インデックス][列インデックス] で要素にアクセス
matrix[0][0] = 1; // 0行0列目に1を代入
matrix[1][2] = 5; // 1行2列目に5を代入
System.out.println(matrix.length); // 出力: 3
System.out.println(matrix[0].length); // 出力: 4
```

### 拡張for文によるループ処理

```
int[] numbers = {10, 20, 30, 40, 50};
for (int num : numbers) {
    System.out.println(num);
}
```

### 2次元配列の初期化

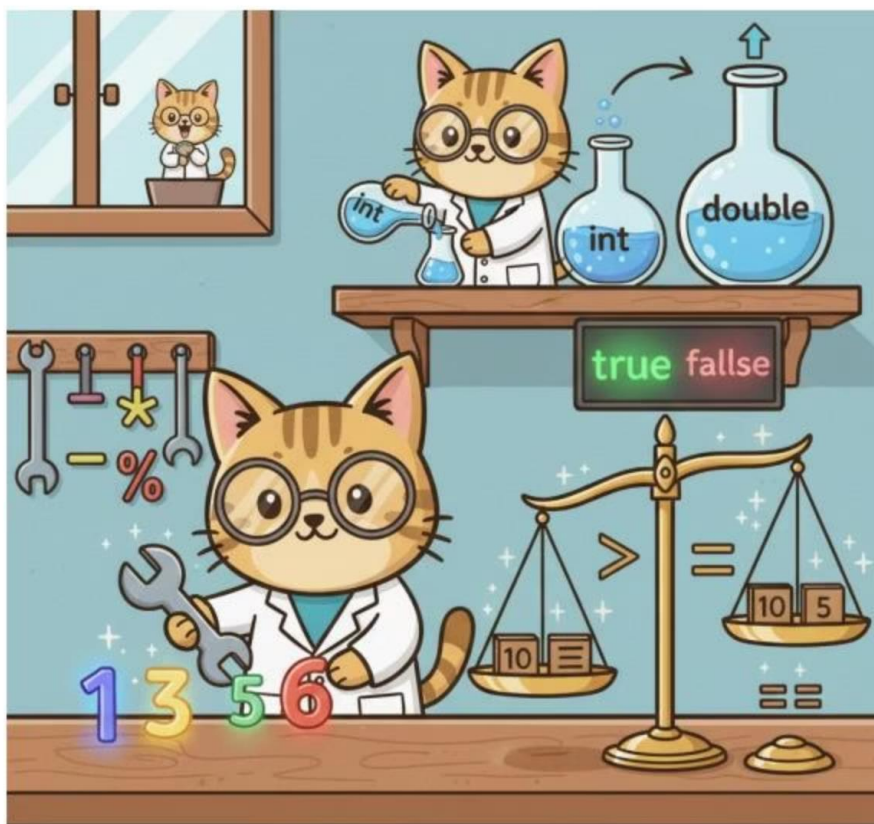
```
int[][] matrix2 = {
    {1, 2, 3}, // 0行目
    {4, 5, 6}, // 1行目
    {7, 8, 9}  // 2行目
};
System.out.println(matrix2[1][1]); // 出力: 5
```



## 第2章のまとめ

# データを入れる魔法の箱!

- 変数は、データを入れておくための名前付きの箱。
- データ型は、その箱にどんな種類のデータが入るかを定めるもの。
- まずは基本の5つを覚えよう！  
int (整数), double (小数), boolean (真偽), char (1文字), String (文字列)
- = は代入を意味する。
- final を付けると、値を変更できない 定数 になる。
- 配列は、同じ型の変数ロッカー。
- 多次元の配列もできる。



## 第3章 計算と比較はお任せ!

### 演算子と型キャスト

この章の目標は、変数に代入したデータを、足したり引いたり、大小を比べたりする方法を学ぶこと。

計算の主役である「演算子」の様々な種類と使い方をマスターし、異なるデータ型同士で計算する際の「型キャスト」という重要なルールを理解します。プログラミングにおける数学の力を身につけましょう。



# 演算子の種類を知ろう

演算子とは、計算や比較など、何らかの処理を行うための記号のことです。  
Javaには様々な演算子が用意されています。



## 算術演算子 (+, -, \*, / ...)

足し算や引き算などの数学的な計算を行います。



## 論理演算子 (&&, || ...)

複数の条件を組み合わせて、より複雑な判定を行います。



## 関係(比較)演算子 (==, >, < ...)

等しいか、大きいかなどを比較し、true/falseを返します。



## 代入演算子 (=, += ...)

変数に値を代入したり、計算結果を代入します。



# 型キャストと演算子の実践

## 主要な演算子

### 算術演算子

|   |        |
|---|--------|
| + | 加算     |
| - | 減算     |
| * | 乗算     |
| / | 除算     |
| % | 剰余(余り) |

### 比較演算子

|    |       |
|----|-------|
| == | 等しい   |
| != | 等しくない |
| >  | より大きい |
| <  | より小さい |
| >= | 以上    |
| <= | 以下    |

### 代入演算子

|    |      |
|----|------|
| =  | 代入   |
| += | 加算代入 |
| -= | 減算代入 |
| *= | 乗算代入 |
| /= | 除算代入 |

### 論理演算子

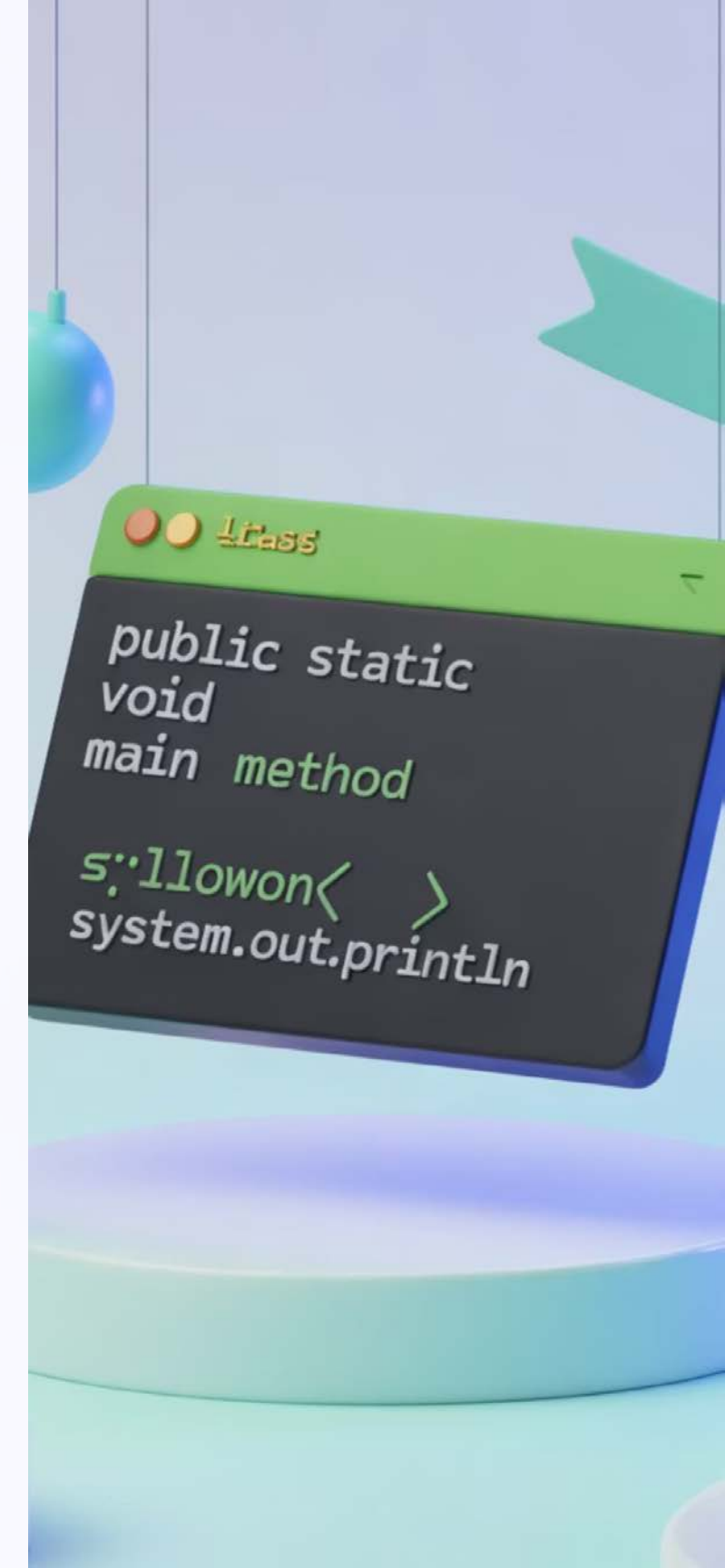
|    |              |
|----|--------------|
| && | AND(両方true)  |
|    | OR(どちらかtrue) |
| !  | NOT(反転)      |

### インクリメント・デクリメント

|    |         |
|----|---------|
| ++ | インクリメント |
| -- | デクリメント  |

# 型キャストと演算子の実践 使用例

```
int a = 10;
a = a + 5;
a = a - 3;
a += 20;
double d = 1.25;
int age = 25;
boolean hasLicense = true;
// 年齢が20歳以上 "かつ" 免許を持っているか?
boolean canDrive = (age >= 20) && (hasLicense == true);
System.out.println(canDrive);
// 結果: true
```



# 型キャストと演算子の実践

## 型キャストの基本

異なる型同士の計算では、より表現能力の高い(大きい)型に自動的に合わせてから計算されます。

### 自動変換（拡大変換）

```
int myInt = 10;
double myDouble = myInt; // 自動変換
// 結果: 10.0 double = int → double
int a = 10;
double b = 3.5;
double result = a + b; // 自動変換
// 結果: 13.5 int+double → double
System.out.println(10.0 / 3); // 自動変換
// 結果: 3.333... doubl/int → double
```

### 手動変換（縮小変換）

```
double myDouble = 9.78;
int myInt = (int) myDouble; // 強制変換
// 結果: 9 (小数点以下は切り捨て)
System.out.println(10 / 3);
// 結果: 3(int同士の割り算は小数点以下が切り捨て)
```

## 使用例

```
int count = 0;
count++; // count が 1 になる
count++; // count が 2 になる
System.out.println(count); // 結果: 2
```

① 少しトリッキーな話（前置と後置） ++を前に置くか(前置: ++x)、後ろに置くか(後置: x++)で、代入や比較と組み合わせた際の評価の順番が変わります。

```
int count = 0;
System.out.println(count++); // 結果: 0
// ←現在の式を評価した後に、インクリメント
System.out.println(count); // 結果: 1
System.out.println(++count); // 結果: 2
// ←インクリメントした後に、現在の式を評価
```

これらの演算子を組み合わせることで、複雑な計算や判定を行うことができます。次の章では、これらを活用して条件分岐を学んでいきます。

## 第3章のまとめ

# 計算と比較はお任せ!

- 異なる型同士の計算では、大きい型に自動で揃えられる（暗黙の型キャスト）。
- 大きい型から小さい型への変換は (型) を使って明示的に行う（明示的な型キャスト）。
- 算術演算子 (+, -, /, %...) で計算ができる。
- 関係演算子 (==, >, !=...) で比較ができ、結果はbooleanになる。
- 論理演算子 (&&, ||) で複雑な条件を組み立てられる。



# 第4章 プログラムの流れをコントロールする

## 条件分岐 if文

この章の目標は、プログラムの実行の流れを、状況に応じて変える方法を学ぶこと。

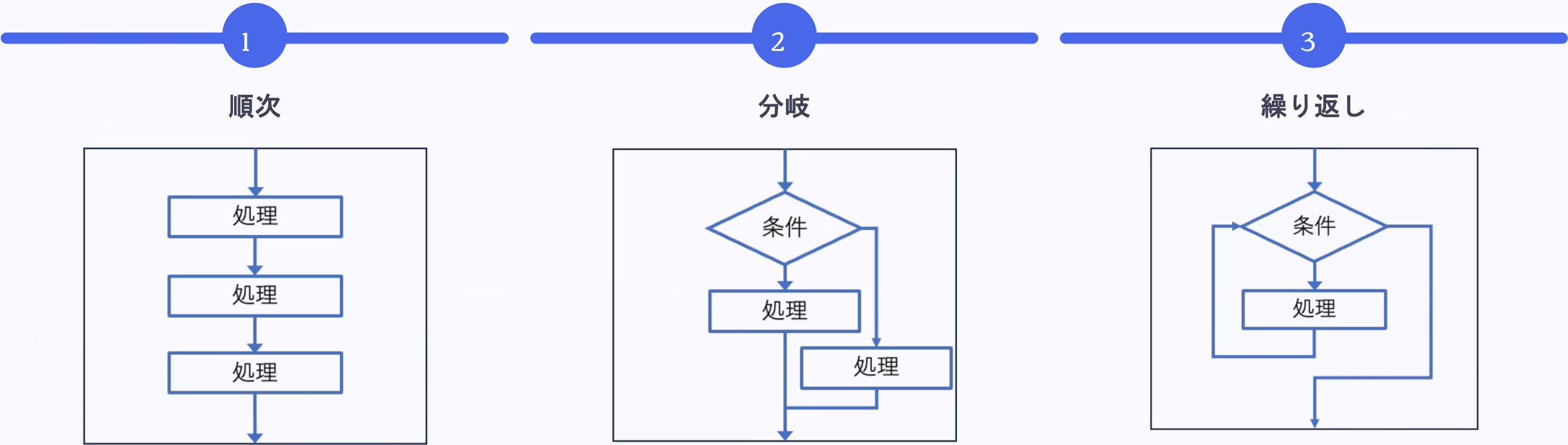
前回学んだ比較の結果（true or false）を使い、プログラムに「もし〇〇だったら、この処理をする」という判断をさせるための「if文」をマスターします。



# 制御文てな～に？

プログラミングの制御文とは、プログラムの実行順序を通常の上から下への流れから変えるための命令です。これによりプログラムは状況に応じて判断したり処理を繰り返したりといった複雑な動作が可能になります。

次の3つはプログラム制御の基本型で、これらを組み合わせることで様々な処理を行うことができます。プログラムは、通常特別な制御文を使わなければ上から下への順次処理で進みます。

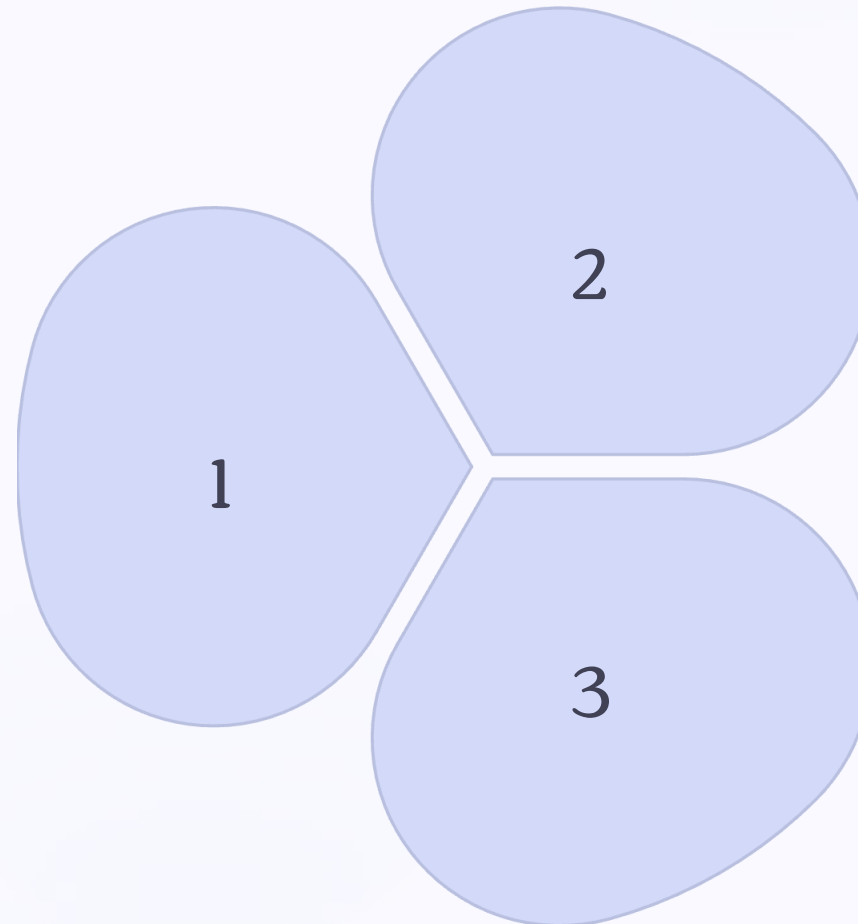


# なぜ判断が必要なの？

プログラムは、状況に応じて実行する処理を変える必要があります。このようなプログラムの流れをコントロールする仕組みを 制御文（せいぎょぶん） と呼び、if文はその代表格です。

## 評価システム

- テストの点数が80点以上なら「合格」、そうでなければ「不合格」と表示する。



## 小売販売システム

- 年齢が20歳未満なら、お酒の購入ボタンを非表示にする。

## SNSシステム

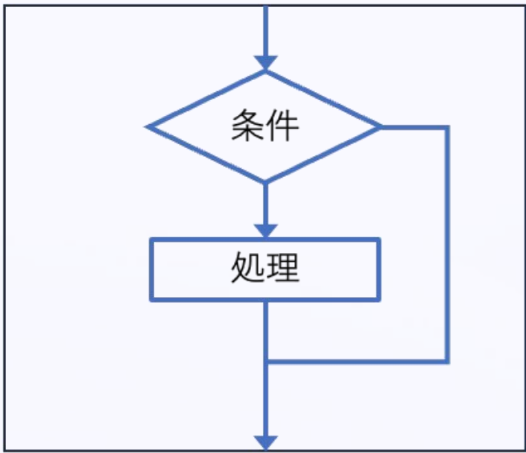
- ログインIDとパスワードが正しければ、マイページに遷移させる。

# if文の基本

## 基本のif文

条件式がtrueの場合にのみ、ブロックの中の処理を実行します。

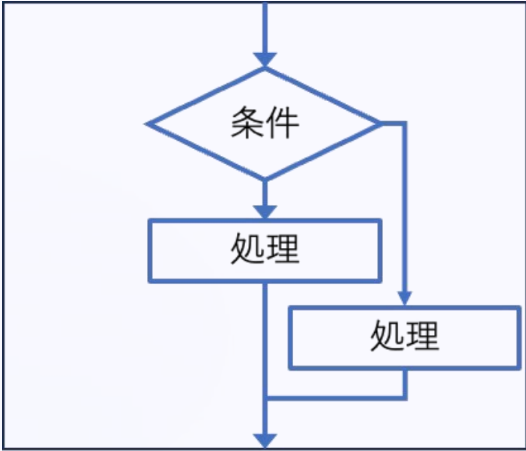
```
if (条件式) {  
    // 条件trueの場合に実行  
}
```



## if-else文

条件がfalseだった場合の処理も書きたいときはelseを使います。

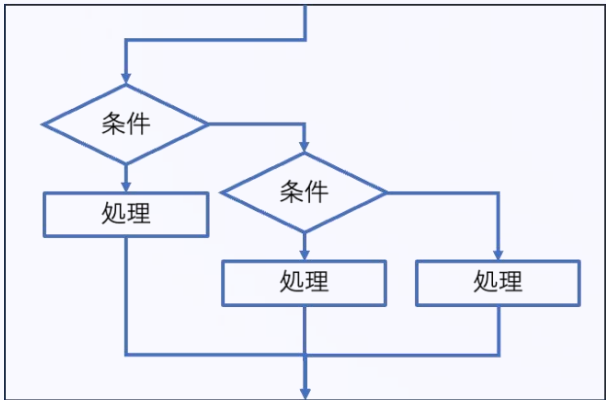
```
if (条件式) {  
    // trueの場合  
} else {  
    // falseの場合  
}
```



## if-else if-else文

選択肢が3つ以上ある場合は、else ifを使って条件を追加できます。

```
if (条件式1) {  
    // 条件式1がtrueの場合  
} else if (条件式2) {  
    // 条件式2がtrueの場合  
} else {  
    // すべてfalseの場合  
}
```



# if文を使ってみよう!



## 処理分岐の例

if文elseを連結して、処理を分岐することが出来ます。

```
int score = 85;
if (score >= 90) {
    System.out.println("素晴らしい! 評価は「優」です。");
} else if (score >= 80) {
    System.out.println("良いですね! 評価は「良」です。");
} else if (score >= 60) {
    System.out.println("もう少し! 評価は「可」です。");
} else {
    System.out.println("残念...評価は「不可」です。");
}
```

**\*\*実行結果:** \*\*良いですね! 評価は「良」です。

なぜ?

score >= 90 (85 >= 90) → false。次へ。

score >= 80 (85 >= 80) → true!

のブロックを実行し、if文全体を終了。

on true

< if >



# if文で注意すべき点



## 中括弧 {} の役割

ifやelseの後に実行する処理が1行だけの場合、中括弧 {} は省略できます。

しかし、非推奨です！後から処理を追加した際に、{}を付け忘れてバグの原因になることが非常に多いです。

```
// 1行なのでOK
if (score >= 80)
    System.out.println("合格です！");

{}を付け忘れると...
if (score >= 80)
    System.out.println("合格です！");
    System.out.println("おめでとう！"); // ←この文がifの範囲外

if (score >= 80) {
    System.out.println("合格です！");
    System.out.println("おめでとう！");
}
```

ルール：if文やelse文を使うときは、処理が1行でも必ず{} を付ける\*\* 習慣にしましょう。

## 第4章のまとめ

# プログラムの流れをコントロールする

- プログラムの流れをコントロールする仕組みを制御文という。
- 制御文の基本型は、順次・分岐・繰り返しの3つ
- if (条件式) は、条件式がtrueのときに処理を実行する。
- else を使うと、条件がfalseだった場合の処理も書ける。
- else if を使うと、複数の条件を繋げて、多くの選択肢を作れる。
- if-else if-elseの連鎖は、最初にtrueになったブロックだけが実行される。
- 処理が1行でも、必ず中括弧{}を付ける癖をつけよう！



## 第5章 もう一つの選択肢！ スマートな分岐 switch文 if文よりもスッキリ書ける場面を知ろう

この章の目標は、if-else if文とは異なるアプローチで条件分岐を行う「switch文」を学ぶこと。

どのような場面でswitch文が適しており、コードをよりクリーンで読みやすくできるのかを理解します。

# if-else ifが読みにくい時

- ある一つの変数の値が「1の場合」「2の場合」「3の場合」...のように、特定の値と一致するかどうかで処理を分けたい場合、if-else ifを何度も繰り返すと、コードが縦に長くなり読みにくくなることがあります。
- 比較が何度も続いていますね。こういう時こそswitch文の出番です！

```
int day = 2;
if (day == 1) {
    System.out.println("Monday");
} else if (day == 2) {
    System.out.println("Tuesday");
} else if (day == 3) {
    System.out.println("Wednesday");
} else {
    System.out.println("Some other day");
}
```

# switch文の基本

## 基本のswitch文

switch文は、()の中の変数の値と一致する\*\*case\*\*を探し、そこから処理を開始します。

```
switch (変数) {  
    case 値1:  
        // 変数の値が値1と一致した場合の処理  
        break;  
    case 値2:  
        // 変数の値が値2と一致した場合の処理  
        break;  
    default:  
        // どのcaseにも一致しなかった場合の処理  
        break;  
}
```

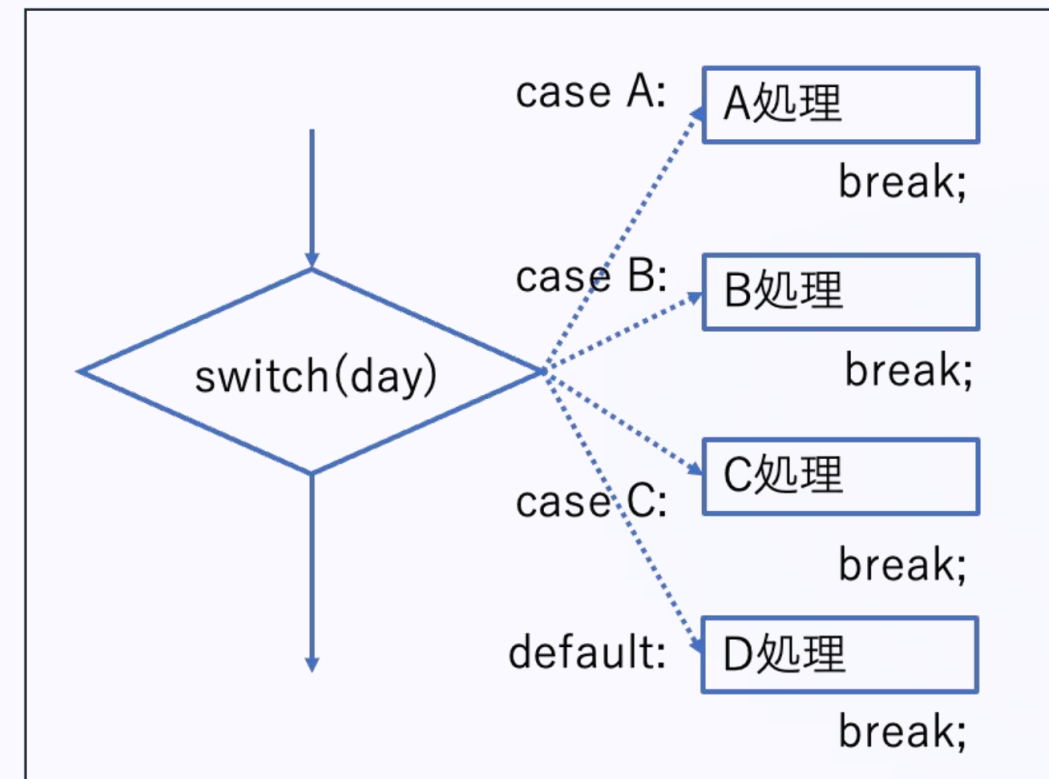
注意点：breakとdefalutに気をつけよう！

case: 比較したい値を指定します。

「もしこの値だったら」という意味です。

break: switch文全体の処理を終了させます。非常に重要です！

default: どのcaseにも当てはまらなかった場合に実行されます。  
if文の最後のelseに相当します。





# switch文を使ってみよう！（数値）



## 数値で分岐

```
int xx = 3;
switch (xx) {
    case 1:
        System.out.println("ONE");
        break;
    case 2:
        System.out.println("TWO");
        break;
    case 3:
        System.out.println("THREE");
        break;
    default:
        System.out.println("OTHER");
        break;
}
```

**\*\*実行結果：\*\*THREE**



# switch文を使ってみよう！（文字列）



## 文字列で分岐

```
// switch文はintだけでなくString型変数にも使える！  
String fruit = "orange";  
switch (fruit) {  
    case "apple":  
        System.out.println("Selected fruit is an apple.");  
        break;  
    case "orange":  
        System.out.println("Selected fruit is an orange.");  
        break;  
    default:  
        System.out.println("Unknown fruit.");  
        break;  
}
```

**\*\*実行結果：\*\***

Selected fruit is an orange.

→ if (fruit.equals("...")) を繰り返すよりも、  
はるかに直感的ですね。



# switch文で注意すべき点（break文）



## breakを忘れると...?（フォールスルー）

caseにbreakがない場合、処理は下のcaseへと突き抜けて実行され続けます。  
これをフォールスルーと呼びます。

```
int xx = 3;
switch (xx) {
    case 1:
        System.out.println("ONE");
    case 2:
        System.out.println("TWO");
    default:
        System.out.println("OTHER");
}
```

意図的にフォールスルーを使い、「3の場合\*\*と\*\*4の場合に同じ処理をしたい」という書き方もできます。

```
case 3:
case 4: // 3と4で同じ処理
    System.out.println("THREE or FOUR");
    break;
...
```

## switch文で注意すべき点（default文）



### default節は付けましょう！

default節は、文法的には省略可能です。しかし、常を書くことを強く推奨します。なぜ？

1. 想定外の値への備え: **case**で想定した以外の値が来たときに、プログラムが何もせずに通り過ぎるのを防ぎます。エラーメッセージを出すなど、意図した処理を行えます。
2. 可読性の向上: これを読んだ他のプログラマが、「想定外の値のことは、ちゃんと考えてあるな」と安心できます。

**default:**

```
// 例えばエラーを記録するなどの処理
```

```
System.out.println("Invalid day: " + day);
```

```
break;
```

# if vs switchどちらを使うべき？

## if文が得意なこと

- 範囲の比較 (`score >= 80`)
- 複雑な条件 (`age >= 20 && hasLicense == true`)
- boolean値そのものの分岐 (`if (isLoggedIn)`)

## switch文が得意なこと

- 一つの変数が、特定の値（定数）と一致するかどうかの比較
- 選択肢（case）が多く、分岐の条件がシンプルな場合

結論： 複雑な条件ならif文。

一つの変数に対するシンプルな多分岐ならswitch文を検討しよう！



## 第5章のまとめ

# もう一つの選択肢！スマートな分岐 switch文

- switch文は、一つの変数に対する特定の値での多分岐が得意。
- case で値を指定し、処理の最後には **break** を忘れないようにする。
- breakを意図的に省略し、フォールスルーを利用するテクニックもある。
- どのcaseにも合致しない場合の処理は **default** に書く。（省略せず、常に書くのが望ましい）
- 複雑な条件や範囲指定はif文、シンプルな多分岐はswitch文と使い分けよう



## 第6章 面倒な作業は自動化！ループ処理 for, while

コンピュータの真骨頂、繰り返し処理を学ぼう

この章の目標は、同じまたは類似した処理を何度も繰り返し実行する方法を学ぶこと。

プログラミングを非常にパワフルにする「ループ（繰り返し）処理」の代表格であるwhile文とfor文をマスターし、面倒な反復作業をコンピュータに任せるスキルを身につけます。

# なぜループが必要なの？

もし、「Hello」と5回表示したい場合、どう書きますか？

```
System.out.println("Hello");  
System.out.println("Hello");  
System.out.println("Hello");  
System.out.println("Hello");  
System.out.println("Hello");
```

問題点：

- 面倒くさい：100回や1万回だったら？
- 修正が大変：「Hello」を「Hi」に変えるのに5箇所も直す必要がある。
- 柔軟性がない：実行する回数を変えるのが難しい。

ループを使えば、これらの問題を一挙に解決できます！

# 条件がtrueの間ずっと：while文

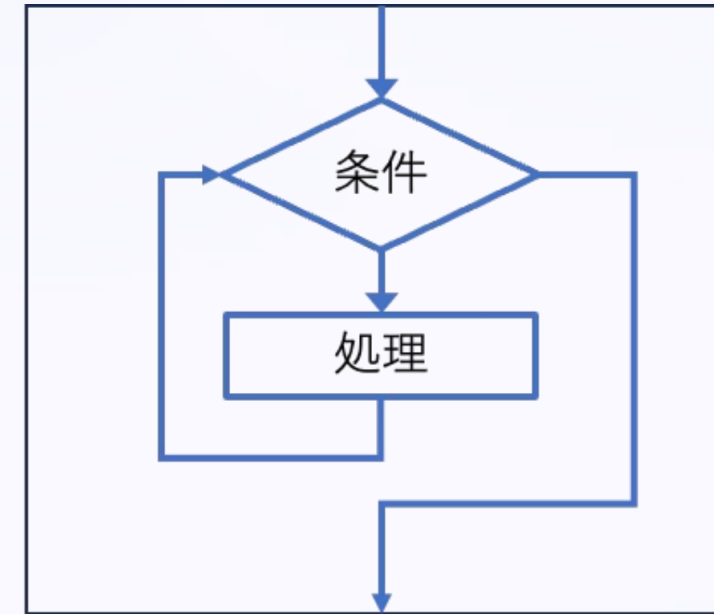
## while文

()の中の条件式がtrueである限り、{}ブロックの中の処理を繰り返し実行します。

```
while (条件式) {  
    // 条件式がtrueの間、繰り返される処理  
}
```

例：「Hello」と5回表示する

```
int i = 0; // ①カウンターを初期化  
while (i < 5) { // ②条件をチェック  
    System.out.println("Hello");  
    i++; // ③カウンターを更新  
}
```



ループ内で条件式のtrue/falseが変化するようにな処理（i++など）を書き忘れると、ループが永遠に終わらない無限ループに陥るので注意！

# 最低1回は実行したい：do-while文

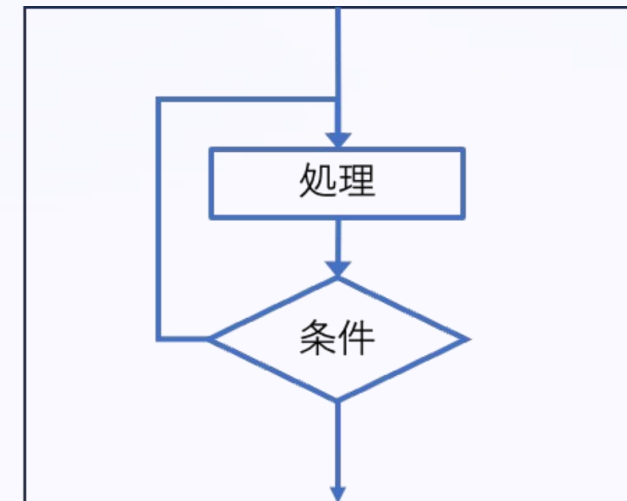
## do-while文

while文と似ていますが、条件のチェックを最後に行うのが特徴です。そのため、ブロック内の処理が最低でも1回は必ず実行されます。

```
do {  
    // 最初に必ず1回実行され、その後条件式がtrueの間繰り返される処理  
} while (条件式);
```

例：「Hello」と5回表示する

```
int i = 5; // ①カウンターを初期化  
do {  
    System.out.println("このメッセージは1回だけ表示されます。");  
    i++; // ②カウンターを更新  
} while (i < 5); // ③条件をチェック
```



# 回数が決まっている場合に最適：for文

## for文

特に決まった回数だけ処理を繰り返したい場合に非常に便利なループです。「カウンターの初期化、条件式、カウンターの更新」を1行で書けます。

```
for (①初期化; ②条件式; ③更新処理) {  
    // 条件式が true の間、繰り返される処理  
}
```

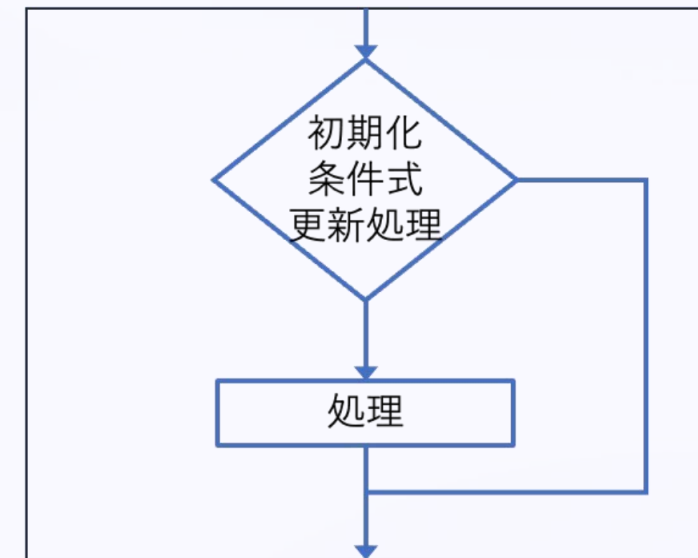
例：「Hello」と5回表示する

Java

```
for (int i = 0; i < 5; i++) {  
    System.out.println("Hello");  
}
```

実行順序：

① → (② → 中身 → ③) → (② → 中身 → ③) → ... → ②が**false**になったら終了



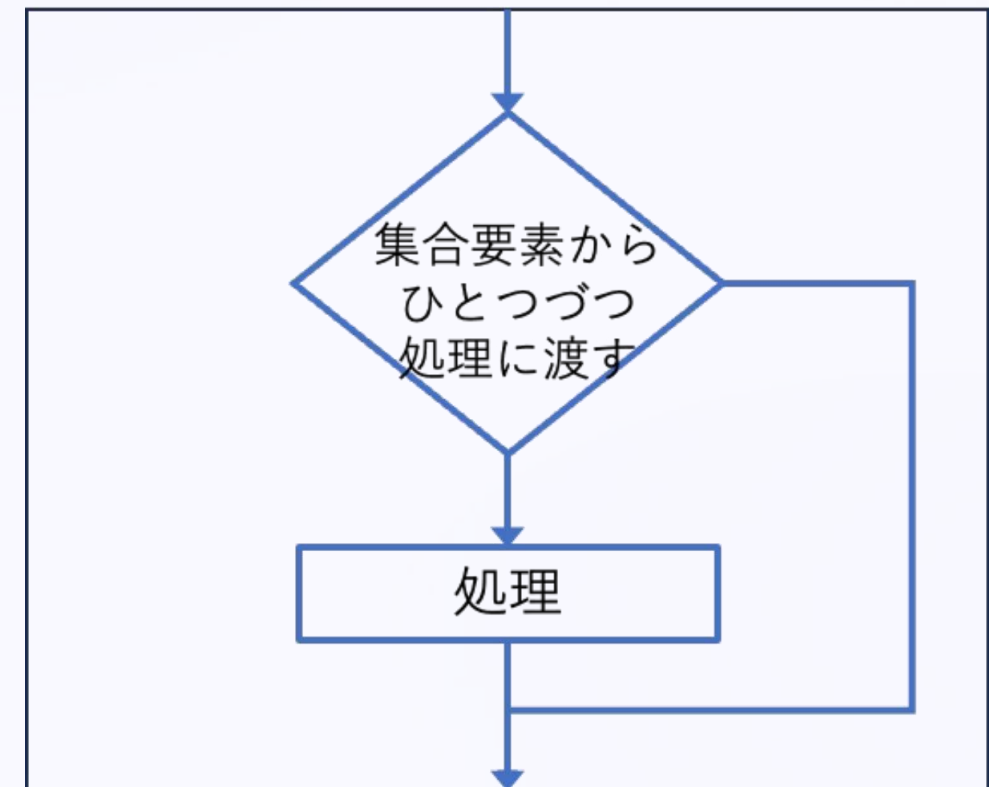


# 集合の要素に最適：拡張for文

## 拡張for文

配列やコレクション（複数のデータのかたまり）のすべての要素に対して、順番に処理を行いたい場合、よりシンプルに書ける拡張for文：for-each文があります。

```
// 配列（同じ型のデータのリスト）  
char[] array = {'A', 'B', 'C'};  
  
// 拡張for文  
// arrayの中から、1つずつ要素を取り出してchに入れ、  
// ブロックを実行する  
for (char ch : array) {  
    System.out.println(ch);  
}
```



カウンター変数(i)が不要で、シンプルに書けます。

# ループの制御：break文

## 途中で中止したい：break文

break文は、switch文だけでなくループ処理でも使えます。ループの条件に関わらず、ループを即座に強制終了させます。

例：0から9まで表示するが、もし*i*が4になったら途中でループを抜けます

```
for (int i = 0; i < 10; i++) {  
    if (i == 4) {  
        break; // ループを抜ける！  
    }  
    System.out.println(i);  
}
```

実行結果：

0  
1  
2  
3



# ループの制御：continue文

## スキップしたい：continue文

continue文は、ループを終了させるのではなく、その回の残りの処理をスキップして、次の回のループにジャンプします。

例：0から9まで表示するが、iが5の時だけスキップする

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        // 今回の処理は飛ばして、次のループへ！  
        continue;  
    }  
    System.out.println(i);  
}
```

実行結果：

0 1 2 3 4 6 7 8 9

(5だけが表示されていない)



## 第6章のまとめ

# 面倒な作業は自動化！ループ処理 for, while

- 同じ処理を繰り返す仕組みをループと呼ぶ。
- while文は、条件がtrueの間、処理を繰り返す。無限ループに注意が必要。
- for文は、決まった回数の繰り返しに最適で、カウンターの管理が1行で書ける。
- breakはループを即座に中断する。
- continueは、その回の処理だけをスキップして次に進む。





## 第7章 コードを整理整頓！再利用できる部品「メソッド」

プログラムを部品化して、見通しを良くしよう

この章の目標は、処理をひとまとまりの「部品」として定義し、名前を付けて何度も呼び出せるようにする「メソッド」の概念を理解すること。

メソッドを使うことで、コードの重複をなくし、プログラム全体の構造をクリーンで管理しやすくするスキルを身につけます。

# 同じコード、何度も書いていませんか？

同じコード、何度も書いていませんか？

例えば、プログラムの様々な場所で、自己紹介を表示する必要があったとします。

```
// 処理A
System.out.println("--- Profile ---");
System.out.println("Name: John");
System.out.println("Age: 25");

// ... 何か別の処理 ...
// 処理B
System.out.println("--- Profile ---");
System.out.println("Name: John");
System.out.println("Age: 25");
```

問題点：

- コードの重複：同じコードが複数箇所に存在している（DRY原則違反）。
- 修正が大変：年齢が変わったら、すべての箇所を修正する必要がある。



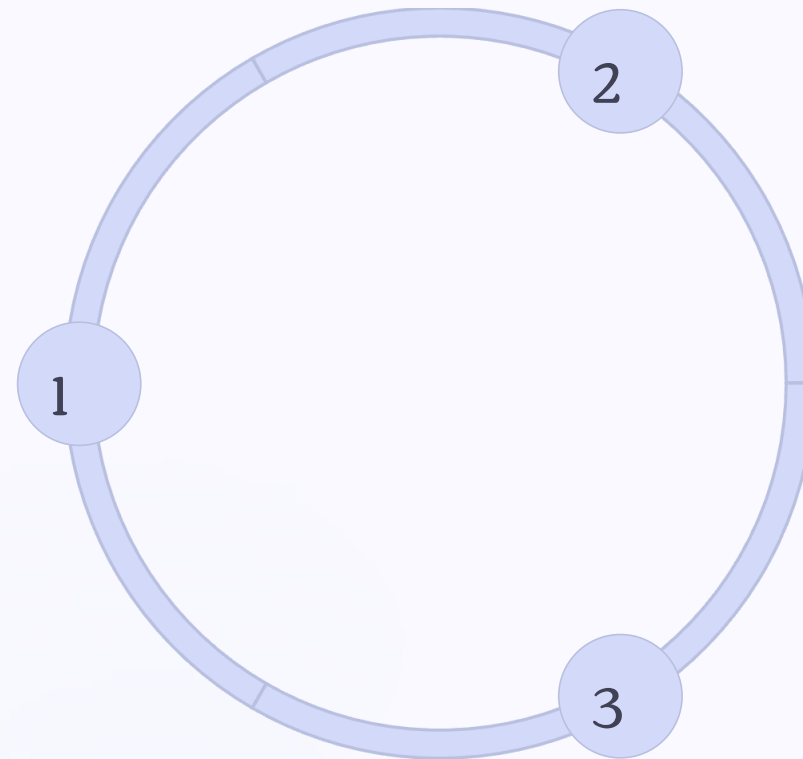
# メソッドとは？

メソッドとは、特定の処理をひとまとめにし、名前を付けたコードのブロックです。

料理のレシピに例えるなら、一つ一つの手順ではなく、「野菜を切る」「ソースを作る」といった、まとまった作業単位に名前を付けておくようなものです。

## 再利用性

一度作れば、名前を呼ぶだけで何度でも同じ処理を実行できる。



## 可読性

mainメソッドがシンプルになり、プログラム全体の流れが追いやすくなる。

## 保守性

修正が必要な場合、そのメソッドの中身を1箇所直すだけで済む。

# メソッドの定義

**メソッドの定義** : `public static` 戻り値の型 メソッド名 (引数1, 引数2, ...)

**戻り値の型** :

メソッドは処理の結果を、呼び出し元に値として返却することが出来ます。戻り値の型とはその返却される値の変数型になります。何も結果を返さない（実行するだけ）であれば、`void` 型とします。

**メソッド名** :

メソッド名は、メソッドがどんな役割をするか他の名前と被らないユニークな名前を付けます。

**引数 (ひきすう)** :

引数はパラメータとも呼ばれ一つの引数は "**変数型 引数名**" の形式です。引数はなくても良いですが、複数ある場合は、それぞれに','カンマ区切りで表現します。呼び出し元が何らかの値を渡したい時に定義し、メソッド側で渡された引数の値に従って処理を変えたり計算したりに使います。

- 今はメソッドの戻り値の前に、**public**と**static**という2つのキーワードを付けてください。**public**は公開されたメソッドの意味で、他のクラスのメソッドからも呼べるという意味。また、**static**が付くと同じ**static**の付いたメソッドの**main**メソッドなどからも直接呼ぶことが出来ます。  
**public static**の詳しくは？このキーワードの真価は、Javaの最も重要な概念である\*\*オブジェクト指向プログラミング (OOP) \*\*を学ぶことで明らかになります。今は「おまじないのようなもの」と考えて、自作のメソッドにも**public static**を付けて先に進みましょう！

# メソッドの定義と呼び出し例

## メソッド定義

下記はMainクラス内に、メソッド定義した例です。

1

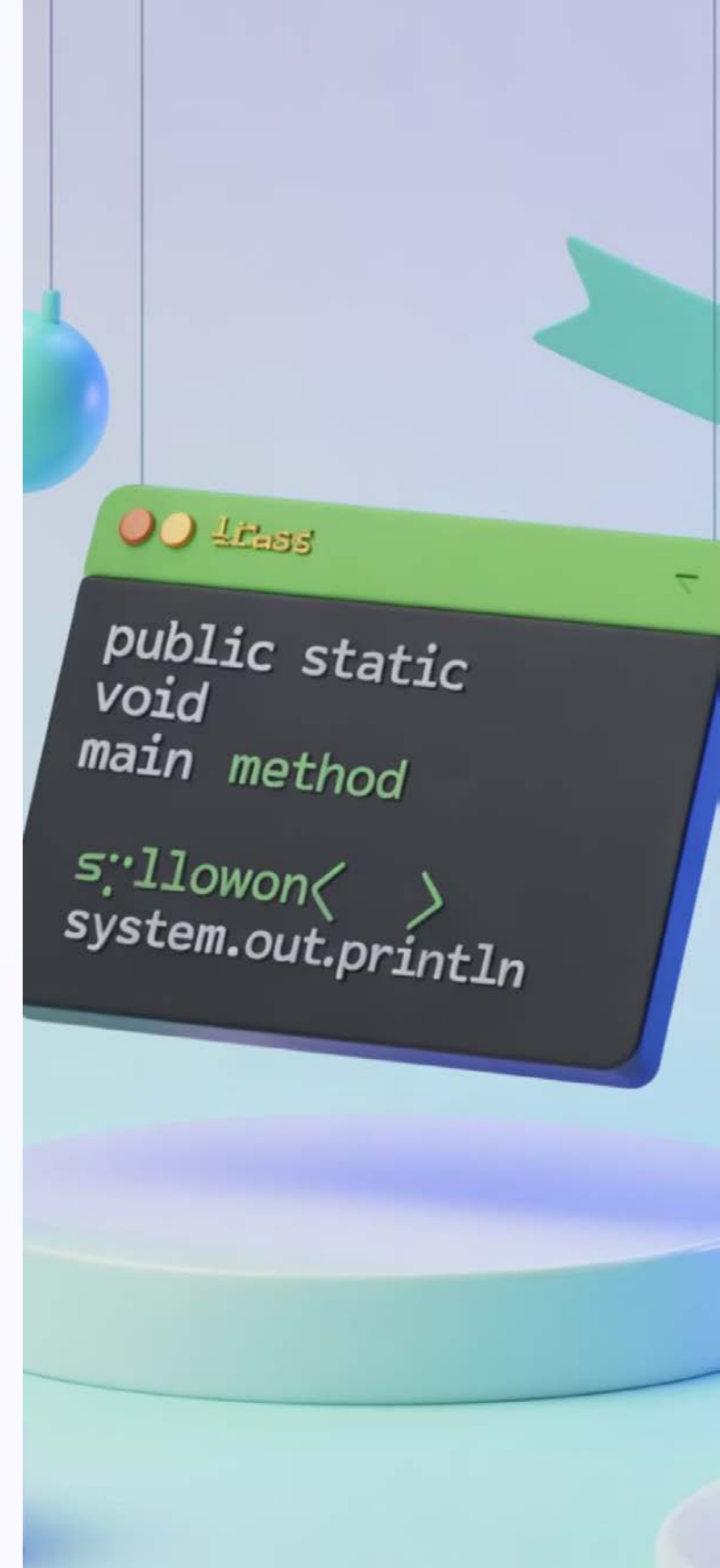
```
public class Main {  
    // ① メソッドの定義 (部品の作成)  
    static void printProfile() {  
        System.out.println("--- Profile ---");  
        System.out.println("Name: John");  
        System.out.println("Age: 25");  
        System.out.println("-----");  
    }  
    // 下記に続く...
```

## メソッド呼び出し

定義したメソッドをmainメソッドから呼び出します。

2

```
public static void main(String[] args) {  
    System.out.println("処理を開始します。");  
    // ② メソッドの呼び出し (部品の使用)  
    printProfile();  
    System.out.println("処理を終了します。");  
}
```



# メソッドに情報を渡す（引数）



## 情報を渡す

メソッドは、呼び出し元からデータを受け取ることができます。この受け取るための変数を\*\*引数（ひきすう）\*\*と呼びます。（色々な人のプロフィールを表示できる、汎用的なメソッドになりました！）

```
public class Main {  
    // メソッドの定義  
    static void printUserProfile(String name, int age) {  
        System.out.println("--- Profile ---");  
        System.out.println("Name: " + name + " Age: " + age);  
    }  
    public static void main(String[] args) {  
        // メソッドを呼び出す際に、具体的な値を渡す  
        printUserProfile("Alice", 30);  
        printUserProfile("Bob", 22);  
    }  
}
```

結果：

```
--- Profile ---  
Name: Alice Age: 30  
--- Profile ---  
Name: Bob Age: 22
```



## メソッドから結果を受け取る（戻り値）



### 結果を受け取る

メソッドは、処理の結果を呼び出し元に返すこともできます。この返される値を戻り値（もどりち）または戻り値と呼びます。**return**：メソッドの処理を終了し、指定した値を返します。

```
public class Main {  
    // メソッドの定義  
    static int add(int x, int y) {  
        int result = x + y;  
        // `return` で結果を呼び出し元に返す  
        return result;  
    }  
    public static void main(String[] args) {  
        // メソッドを呼び出す際に、具体的な値を渡す  
        int answer = add(5, 3); // 戻り値を変数で受け取る  
        System.out.println("5 + 3 = " + answer);  
        int anotherAnswer = add(100, 200);  
        System.out.println("100 + 200 = " + anotherAnswer);  
    }  
}
```

結果：

5 + 3 = 8

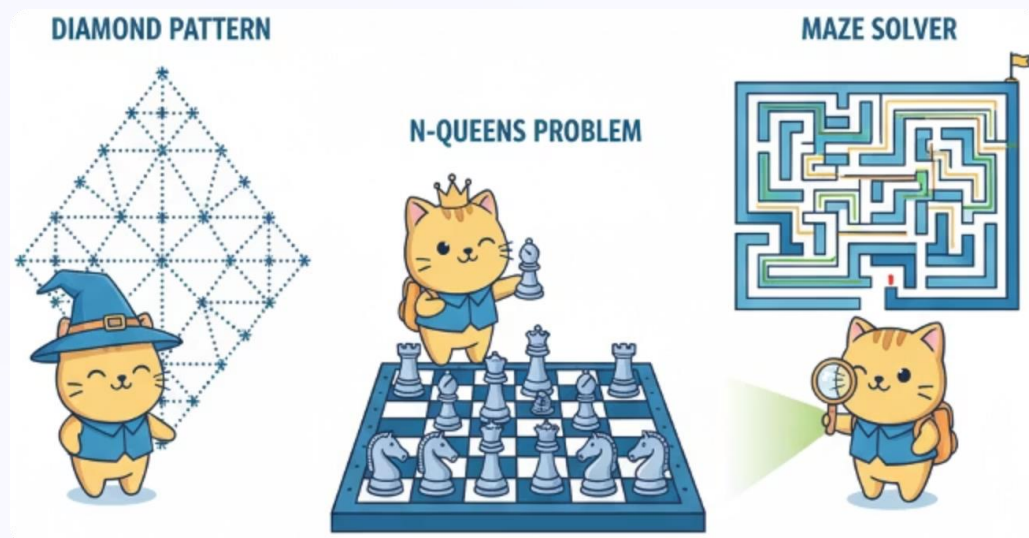
100 + 200 = 300



## 第7章のまとめ

# コードを整理整頓！再利用できる部品「メソッド」

- メソッドとは、処理をまとめた再利用可能な部品のこと。
- `void` は、メソッドが戻り値を返さないことを示す。
- 引数を使うと、メソッドに外部から情報を渡せる。
- `return` を使うと、メソッドが呼び出し元に結果（戻り値）を返せる。
- メソッド化により、コードの重複をなくし、可読性と保守性を向上させることができる。



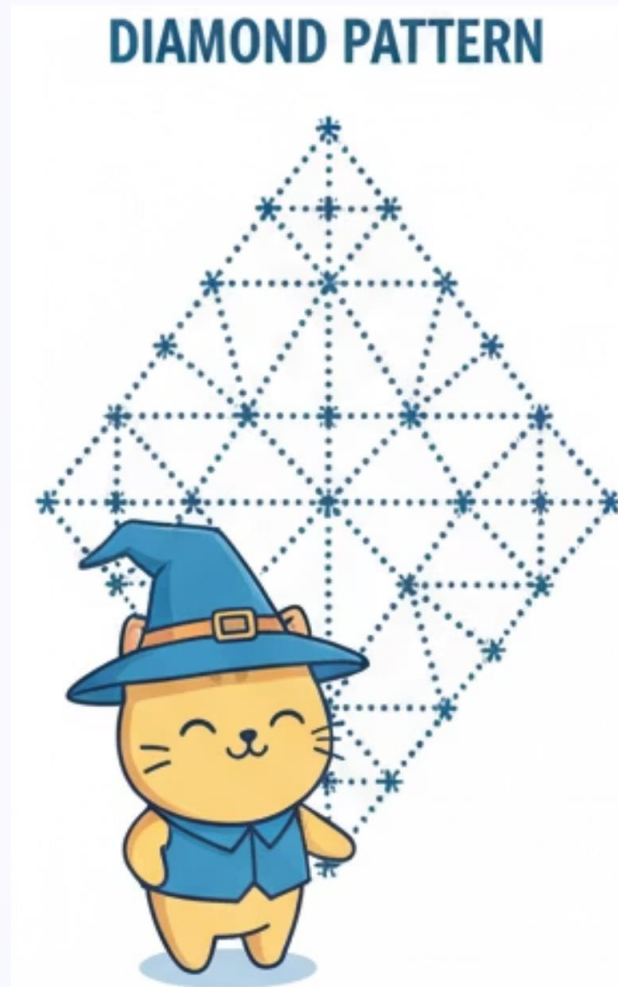
## 第8章 チュートリアル

課題をプログラミングしてみよう

この章の目標は、提示された課題を、問題を単純化したものから3つのステップで徐々に肉付けしてプログラムを完成させましょう。

# 課題 1 : ダイヤモンド表示問題

## 第8章 チュートリアル



問題 :

コンソール画面のキーボードから、幅サイズを入力してそのサイズに合ったダイヤモンド形となるように、\* (アスタリスク) で表示してみよう！

Step 1 : \* で指定されたサイズの四角形を作ろう！

Step 2 : \* で上半分の三角形を作ろう！

Step 3 : \* でダイヤモンド形を作ろう！ (完成形)

## 課題 2 : Nクイーン問題

### 第8章 チュートリアル



問題：

コンソール画面のキーボードから、サイズ $N$ を受け取り $N \times N$ のチェスの盤面を作り $N$ 個の女王がお互いに取られない様な位置に配置する全部のパターンを答えなさい。（女王：クイーン駒は、縦横斜め真っ直ぐにどこまでも移動できます。）

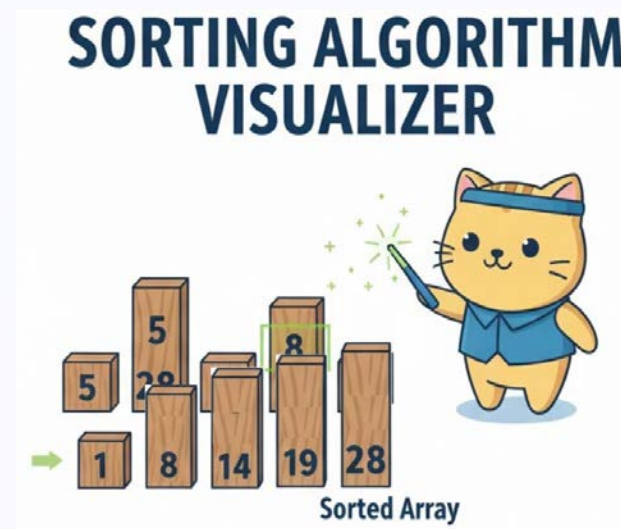
Step 1 : ユーザーから盤面のサイズ $N$ を受け取り $N \times N$ の盤面を作成し  
任意の場所にクイーンを配置して表示する。

Step 2 : 指定したマス(行, 列)に女王を  
置けるかどうかを判定するロジックを実装する。

Step 3 : Step2を繰り返して、  
女王が置ける場所の全パターンを表示する（完成）

# 課題 3 : ソート問題

## 第8章 チュートリアル



問題：

コンソール画面のキーボードから整数値 $N$ を受け取り、テストデータとして $N$ 個整数配列に、 $1 \sim N$ の連続整数値をランダムな場所に格納する。配列の順に格納された整数値が‘\*’（アスタリスク）の数になるように左から縦方向の棒グラフとして並べて表示します。その後そのグラフをソートして左から低い順に並べかえて表示しなさい。

Step1: ユーザーから整数値 $N$ を受け取り、整数配列に  
 $1 \sim N$ の連続整数値をランダムな場所に格納する。

Step2: 整数配列にある値を‘\*’（アスタリスク）の数として棒グラフを作成

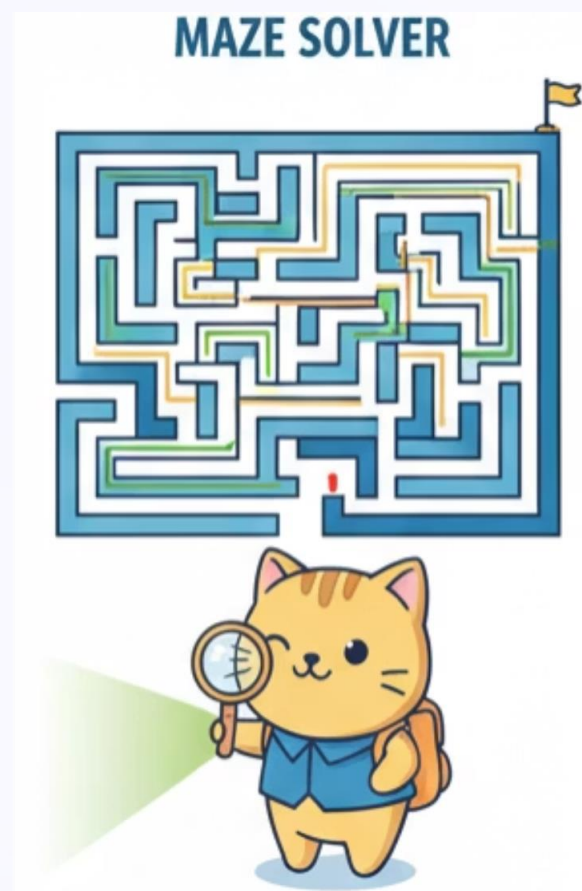
Step3: ソートした結果を棒グラフを表示する（完成）

色んなソートの解放（アルゴリズム）例を載せておきます。



## 課題 4 : 迷路問題

### 第8章 チュートリアル



問題：

与えられた縦横サイズの迷路で、左上のS（スタート）から右下のG（ゴール）へ正しく迷路を突破するルートを探索して表示しなさい。

尚、迷路作成モジュールは先に与えられるものとする。（MazeGenerator.java）

徐々に問題が解けるよう、ステップを自分で考えてみましょう！

ヒント：壁を右か左のどちらかに沿って進んでいく

これはかなり難しいので今は出来なくてもOKです。

いろんな解法（アルゴリズム）の例を載せておきます。

なお、アルゴリズムの学習をするとプログラミングスキルが上がります。ソート（並べ替え）アルゴリズムや、サーチ（探索）アルゴリズムなども勉強してみましょう！